

Update a Custom UI

Information last updated 28 ene 2025

SME	Jan Rusch
EPIC	CIC Custom User Interface and Custom Development

- 1  Instructions
 - 1.1  Custom UI Upgrade Process Flow
 - 1.2  Prerequisites
 - 1.3 **Note:** This process requires careful planning and testing to avoid breaking existing functionality.
 - 1.4  Step 1: Prepare for Upgrade
 - 1.4.1 1.1 Create a Backup
 - 1.4.2 1.2 Document Your Customizations
 - 1.5  Step 2: Get the Latest Template
 - 1.5.1 2.1 Download New Template
 - 1.5.2 2.2 Prepare Your Project
 - 1.5.3  File Replacement Process:
 - 1.6  Step 3: Restore Basic Configuration
 - 1.6.1 3.1 Update Configuration Files
 - 1.6.2 3.2 Install Dependencies
 - 1.7  Step 4: Regenerate Plugins
 - 1.7.1 4.1 Recreate Your Plugins
 - 1.7.2 4.2 Copy Your Custom Code
 - 1.8  Step 5: Apply Additional Customizations
 - 1.8.1 5.1 Copy Custom Files
 - 1.9  Step 6: Test Your Upgrade
 - 1.9.1 6.1 Run the Application
 - 1.9.2 6.2 Test All Functionality
 - 1.9.3
 - 1.9.4 6.3 Fix Any Issues
 - 1.10  Step 7: Deploy Your Upgrade
 - 1.10.1 7.1 Package Your UI
 - 1.10.2 7.2 Deploy to Studio Modeling
 - 1.11  Additional Resources
 - 1.12  Questions?
- 2  Related articles

Instructions

Hey! Today, I'm going to show you how to upgrade your custom UI to get the latest features and improvements from Hyland Experience.

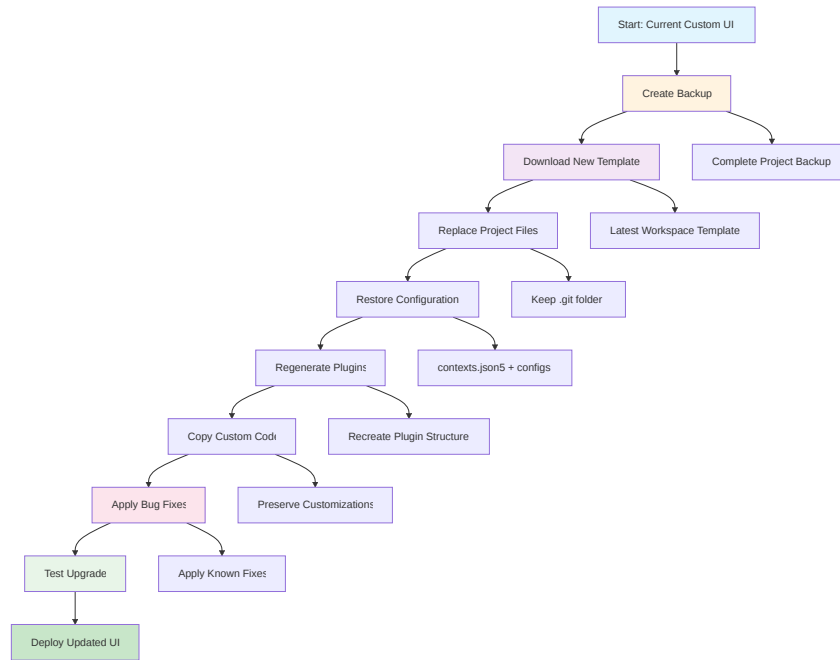
 Custom UIs don't receive automatic updates like the default UI. When you create a custom UI, it's based on a specific version of the workspace template, and you need to manually upgrade it to get new features.

What you'll learn today:

- Why custom UIs need manual upgrades
- How to safely upgrade your custom UI
- How to preserve your customizations
- Common challenges and how to overcome them

Let's upgrade your custom UI!

Custom UI Upgrade Process Flow



Prerequisites

Before we start, make sure you have:

- **A working custom UI** that you want to upgrade
- **Access to the latest workspace template** from Studio Modeling
- **Version control** set up for your project
- **Backup of your current custom UI**
- **Documentation of your customizations**

Note: This process requires careful planning and testing to avoid breaking existing functionality.

Step 1: Prepare for Upgrade

1.1 Create a Backup

CRITICAL: Before starting, create a complete backup of your current custom UI:

```
1 # Create a backup of your current project
2 cp -r your-custom-ui-project your-custom-ui-project-backup
```

 **Git is a fantastic solution for this.**

1.2 Document Your Customizations

Create a list of all your customizations:

- **Custom plugins** and their functionality
- **Modified files** outside of plugins
- **Configuration changes** in Studio Modeling
- **Custom styling**
- **Third-party integrations**

🔄 Step 2: Get the Latest Template

2.1 Download New Template

1. **Go to Studio Modeling** and create a new Custom UI
2. **Choose "Generate from template"** to get the latest version
3. **Download the source code** and extract it
4. **Note the version** and any new features

✓ See [🚀 Hyland Experience Workspace UI Template](#) for How To Create a Custom UI

2.2 Prepare Your Project

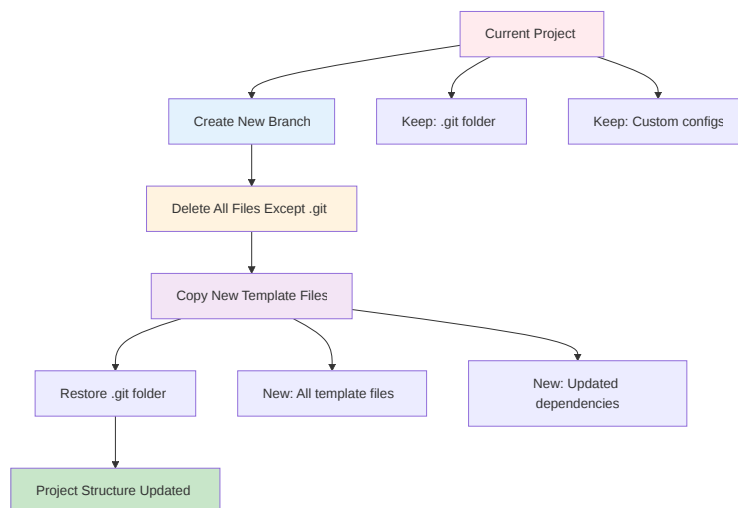
1. **Create a new branch** for the upgrade:

```
1 | git checkout -b upgrade-to-latest-version
```

```
PS C:\Users\jrusch\Downloads\example-ui-utgba> git checkout -b upgrade-to-latest-version
Switched to a new branch 'upgrade-to-latest-version'
PS C:\Users\jrusch\Downloads\example-ui-utgba>
```

2. **Delete everything except .git** from your project folder
3. **Copy all files** from the new template into your project folder

🔄 File Replacement Process:



This will generate many git changes - you may need to commit them to avoid "spawn ENAMETOOLONG" errors.

⚙️ Step 3: Restore Basic Configuration

3.1 Update Configuration Files

1. **Restore your** `config/contexts.json5` from your backup
2. **Update** `project.variables.json5` if needed
3. **Verify environment settings** are correct

3.2 Install Dependencies

```
1 npm ci
2 npm run setenv
```

Step 4: Regenerate Plugins

4.1 Recreate Your Plugins

Run the commands to regenerate your plugins with the same names:

```
1 # Recreate your main plugin
2 npx nx generate @hyland/extend:plugin --name your-plugin-name --addTranslations true
3
4 # Recreate your pages
5 npx nx generate @hyland/extend:page --pluginName plugins-your-plugin-name --pageName your-page-name
6
7 # Recreate your form widgets
8 npx nx generate @hyland/extend:form-widget --pluginName plugins-your-plugin-name --formWidgetName your-widget-name
```

4.2 Copy Your Custom Code

1. **Copy your plugin code** from the backup:

```
1 cp -r your-custom-ui-project-backup/libs/plugins/* libs/plugins/
```

2. **Verify all files** are copied correctly

3. **Check for any conflicts** or missing files

Step 5: Apply Additional Customizations

5.1 Copy Custom Files

Copy any files you modified outside of plugins:

- **README.md** with your customizations
- **Custom styling** files
- **Configuration files** you modified
- **Any other custom code**

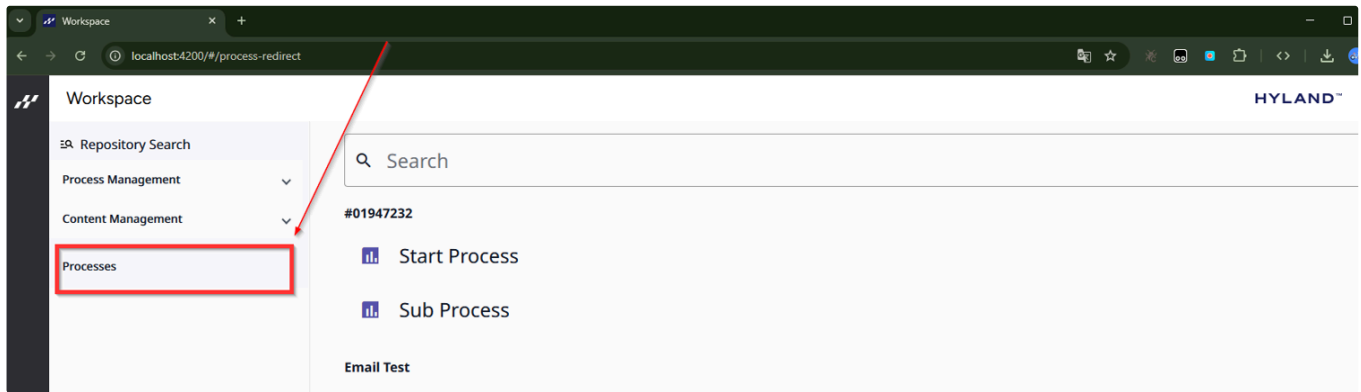
Step 6: Test Your Upgrade

6.1 Run the Application

```
1 npm start workspace-hxp
```

6.2 Test All Functionality

- **Navigation** works correctly
- **Custom pages** load properly
- **Form widgets** function as expected
- **Authentication** works
- **All customizations** are preserved



6.3 Fix Any Issues

- **Check console** for errors
- **Test in different browsers**
- **Verify all features** work as before

Step 7: Deploy Your Upgrade

7.1 Package Your UI

```
1 npm run pack-build workspace-hxp
```

7.2 Deploy to Studio Modeling

1. **Upload the new zip file** to Studio Modeling
2. **Test in development** environment first
3. **Deploy to production** when ready

Additional Resources

- **Hyland Experience Documentation**
- **Plugin Development Guide:** [Creating a Plugin Page]
- **Custom UI Development:** [Creating an Hyland Experience Application \(Custom UI\)](#)

Questions?

Reach out to the team! We're here to help you! 

Thanks, and happy upgrading!

Related articles